

M19 / LEARNER BOOK

Interviews, technical English and job launch

Standalone Zero-to-Job-Ready Data Engineer 2026

14 controlled lessons

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

START HERE How to use this module	3
DE19-01 Truthful one-page Data Engineer resume	4
DE19-02 GitHub and project portfolio final polish	7
DE19-03 LinkedIn/professional profile without overclaiming	9
DE19-04 Degree-aware job targeting: apply/skip rules	11
DE19-05 SQL interview drills	13
DE19-06 Python interview drills	16
DE19-07 Data-engineering concepts and troubleshooting drills	19
DE19-08 Project walkthrough interview	22
DE19-09 Architecture/system-design defense for junior roles	25
DE19-10 Behavioral answers and career-transition story	27
DE19-11 Technical English: explain code, failures and decisions clearly	29
DE19-12 Application tracker, feedback loop and weekly repair cycle	31
DE19-13 Timed coding test: arrays, strings and hash-map problems	33
DE19-14 Combined SQL + Python interview simulation	35

START HERE

How to use this module

Open the matching practice folder from the course root: 18_M19/practice. Work on a copy. Commands printed in a lesson use that module as the working folder unless the lesson names a different location.

Study in lesson order. After the last lesson, use Quick Revision, take the Module Revision Test, then open its separate Protected Answers. Gates are additional assessments at the boundaries shown in the course guide.

Course rule: You do not need to sound like a senior engineer. You do need to be truthful, structured and able to defend the work you actually built. No fake experience, no fake production scale, no degree hiding, and no forced timer during mastery.

This module turns M00-M18 evidence into hiring artifacts and interview behavior. Every lesson produces something reusable: a resume, portfolio audit, profile, job-targeting rubric, saved SQL/Python drills, incident reasoning, project/system-design defense, STAR stories, technical-English recordings, an application feedback loop and cold simulations.

Study route: Work the artifact/task first, validate it, save the genuine attempt, then use the protected review. G07 Gate A is cold. If repair is needed, close Review A and take the different Gate B.

DE19-01

Truthful one-page Data Engineer resume

Watch first

[1. Indeed - Resume Writing: How to Create a Strong Resume](#)

00:32-03:36 - bounded resume structure/credentials/digital-version route

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 1 of 14 | Job-launch evidence, not note-copying | Normal mastery is untimed

Why this matters

A fresher resume is not a biography and it is not a place to manufacture work experience. Its job is to let a reviewer verify, in about a minute, what role you target, what you can actually build, and where the proof lives. A one-page resume is especially useful when your strongest evidence is projects rather than years of employment.

Picture it in your head

Picture the resume as an index into your evidence bundle. Every strong bullet has a subject, an action, a technical boundary, and a result you can defend. If a recruiter circles a claim, you should be able to open a repository, test result, screenshot, run log or design note that supports it.

Words before we use them

- Evidence-backed claim = a statement supported by a project artifact or real experience.
- Scope = the boundary you actually owned or built.
- Outcome = the validated result, not a fabricated business impact.
- ATS-safe = simple text structure that parses without relying on decorative columns or graphics.

Useful visual / current reference

[VIDEO: Indeed - Resume Writing: How to Create a Strong Resume - Open video](#)

Watch: 00:32-03:36 - bounded resume structure/credentials/digital-version route

[OFFICIAL/CURRENT: github_resume](#)

Use rule: The external source is visual/context support. Your saved artifact, query/code, explanation and evidence are authoritative for course assessment. If a video range is not independently bounded, the link stays but is labelled optional rather than invented.

Follow with me once

Start from a deliberately weak bullet: “Built a scalable data pipeline using many technologies.” Rewrite it as: “Built a replay-safe batch pipeline over supplied order/customer/product fixtures; preserved immutable raw evidence, quarantined invalid rows, reconciled source-to-target counts, and documented failure recovery.” Notice what is missing: fake percentages, fake production users, fake years of experience, and claims that the course cannot prove.

Try it yourself

Create your one-page resume from `practice/resume/RESUME_TEMPLATE.md`. Complete `TRUTH_MATRIX_TEMPLATE.csv` before writing bullets. For every project bullet, point to one proof artifact. Education is stated exactly as earned. Skills appear only if you can answer a basic interview question or show project use.

Check your work

- One page at normal font size; no tiny compressed text.
- No “experience” language for a course project unless it is explicitly labelled project experience.
- Every named technology appears in at least one repository or lab.
- No invented performance percentages, users, revenue, company names or employment dates.
- GitHub/profile links are readable and work.

Break case / common failure

A common failure is producing something that sounds polished but cannot survive one follow-up question. Another is hiding uncertainty instead of naming it. Save the weak first attempt, identify the unsupported claim or reasoning gap, and repair the smallest boundary.

Recovery ladder

1) Re-read the exact question/requirement. 2) State grain/contract/evidence before tools. 3) Remove claims you cannot prove. 4) Reproduce the smallest failing example. 5) Repair, rerun and reconcile. 6) Explain the corrected result in plain language without opening the answer file.

Fresh changed retry

Take one project bullet and remove the technology names. Explain the engineering problem and validation first; then add only the tools that materially solved it.

Revision in your own words

Truthful one-page Data Engineer resume: truthful claim -> concrete evidence -> explicit validation -> known limitation -> changed retry. If the answer depends on “trust me,” it is not ready for interview use.

Evidence to keep

Keep the genuine first attempt plus the repaired version. Mastery means you can produce or explain the result without reading the learner book, and can answer at least one “why?”, one failure-mode question and one changed-input question.

DE19-02

GitHub and project portfolio final polish

Watch first

[1. freeCodeCamp - Git & GitHub Crash Course for Beginners \[retained course visual\]](#)

Optional visual - no verified bounded range for profile/portfolio polish

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 2 of 14 | Job-launch evidence, not note-copying | Normal mastery is untimed

Why this matters

A GitHub profile is useful only when a reviewer can understand the work quickly. A pile of repositories with unclear names, tutorial leftovers, secrets, broken setup and no expected result can weaken rather than strengthen a resume.

Picture it in your head

Imagine a reviewer has two minutes. The first screen should answer: what do you build, which 3-5 repositories matter, what problem does each solve, how can it be run, and what evidence proves it works?

Words before we use them

- Profile README = README shown on your GitHub profile when configured correctly.
- Pinned repository = a project deliberately surfaced on the profile.
- Clean clone = a fresh copy that can be run without hidden local files.
- Evidence index = links from README to tests, outputs, screenshots or run logs.

Useful visual / current reference

[VIDEO: freeCodeCamp - Git & GitHub Crash Course for Beginners \[retained course visual\] - Open video](#)

Watch: Optional visual - no verified bounded range for profile/portfolio polish

[OFFICIAL/CURRENT: github_resume](#)

[OFFICIAL/CURRENT: github_readme](#)

Use rule: The external source is visual/context support. Your saved artifact, query/code, explanation and evidence are authoritative for course assessment. If a video range is not independently bounded, the link stays but is labelled optional rather than invented.

Follow with me once

For the ParcelFlow project, make the first README screen show problem, architecture, data grain, quick-start command, expected controls, deliberate failure/recovery and known limitations. Keep the detailed learning journal below that. A reviewer should not need to read 500 lines before finding the project purpose.

Try it yourself

Audit three best projects with `practice/portfolio/PORTFOLIO_AUDIT.md`. Prepare a profile README from `PROFILE_README_TEMPLATE.md`, pin only projects you can defend, remove secrets/generated junk, and test README commands from a clean copy.

Check your work

- 3-5 strongest projects surfaced, not every practice folder.
- Each project has problem -> architecture -> run -> validate -> failure/recovery -> limitations.
- No credentials, tokens, personal secrets or machine-specific absolute paths.
- README commands and expected controls agree with current project files.

Break case / common failure

A common failure is producing something that sounds polished but cannot survive one follow-up question. Another is hiding uncertainty instead of naming it. Save the weak first attempt, identify the unsupported claim or reasoning gap, and repair the smallest boundary.

Recovery ladder

1) Re-read the exact question/requirement. 2) State grain/contract/evidence before tools. 3) Remove claims you cannot prove. 4) Reproduce the smallest failing example. 5) Repair, rerun and reconcile. 6) Explain the corrected result in plain

language without opening the answer file.

Fresh changed retry

Give the README to someone who has not seen the course. Ask them to state the grain, input, output and proof in two minutes. Repair whatever they cannot find.

Revision in your own words

GitHub and project portfolio final polish: truthful claim -> concrete evidence -> explicit validation -> known limitation -> changed retry. If the answer depends on “trust me,” it is not ready for interview use.

Evidence to keep

Keep the genuine first attempt plus the repaired version. Mastery means you can produce or explain the result without reading the learner book, and can answer at least one “why?”, one failure-mode question and one changed-input question.

DE19-03

LinkedIn/professional profile without overclaiming

Watch first

[1. LinkedIn Complete Guide 2026 - Profile, Headline, About, Job Search](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 3 of 14 | Job-launch evidence, not note-copying | Normal mastery is untimed

Why this matters

A professional profile can help discovery, but overclaiming is easy: “Data Engineer” can be read as current professional experience when you are still transitioning. The safer goal is clarity about target role, demonstrated skills and project evidence.

Picture it in your head

Treat headline/About/skills as a searchable summary of the same truth matrix used for your resume. The wording can be confident without pretending a title, employer or production responsibility you have not held.

Words before we use them

- Headline = short professional line displayed near your name.
- About = longer summary of focus, evidence and target.
- Keyword = a role/tool phrase used because it accurately reflects your work, not because it is popular.
- Overclaim = wording that reasonably implies experience or responsibility you cannot substantiate.

Useful visual / current reference

[VIDEO: LinkedIn Complete Guide 2026 - Profile, Headline, About, Job Search - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: linkedin_headline](#)

[OFFICIAL/CURRENT: linkedin_profile](#)

Use rule: The external source is visual/context support. Your saved artifact, query/code, explanation and evidence are authoritative for course assessment. If a video range is not independently bounded, the link stays but is labelled optional rather than invented.

Follow with me once

Weak: “Senior Data Engineer | 5+ years building enterprise lakehouses.” Defensible transition headline: “Data Engineering Portfolio | Python • SQL • Spark • Airflow • Delta/Fabric | Building replay-safe data pipelines.” If you later hold the role professionally, update the wording.

Try it yourself

Use practice/profile/PROFILE_CLAIMS_TEMPLATE.md to draft a headline, About section and skills list. Cross-check every technical keyword against your repository evidence.

Check your work

- Headline states direction/evidence without inventing current employment.
- About section explains transition, strongest engineering evidence and target role in plain language.
- Skills list is smaller than your course index; it contains tools you can defend.
- Resume, GitHub and professional profile tell the same story.

Break case / common failure

A common failure is producing something that sounds polished but cannot survive one follow-up question. Another is hiding uncertainty instead of naming it. Save the weak first attempt, identify the unsupported claim or reasoning gap, and repair the smallest boundary.

Recovery ladder

1) Re-read the exact question/requirement. 2) State grain/contract/evidence before tools. 3) Remove claims you cannot prove. 4) Reproduce the smallest failing example. 5) Repair, rerun and reconcile. 6) Explain the corrected result in plain

language without opening the answer file.

Fresh changed retry

Remove all adjectives such as “expert”, “advanced”, “scalable”, “enterprise”. Add back only terms you can support with a concrete design or result.

Revision in your own words

LinkedIn/professional profile without overclaiming: truthful claim -> concrete evidence -> explicit validation -> known limitation -> changed retry. If the answer depends on “trust me,” it is not ready for interview use.

Evidence to keep

Keep the genuine first attempt plus the repaired version. Mastery means you can produce or explain the result without reading the learner book, and can answer at least one “why?”, one failure-mode question and one changed-input question.

DE19-04

Degree-aware job targeting: apply/skip rules

Watch first

[1. Google India - organise job applications and prepare for interviews](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 4 of 14 | Job-launch evidence, not note-copying | Normal mastery is untimed

Why this matters

Entry-level job descriptions mix true hard gates, preferences and wish lists. Applying to everything wastes time; skipping every role that mentions a technical degree also wastes opportunities. You need a repeatable rule that respects mandatory requirements without self-rejecting from reasonable “or equivalent” roles.

Picture it in your head

Think of each job description as four buckets: hard eligibility, core skill match, evidence gaps, and logistics. Hard gates can stop an application. Skill gaps create a repair plan. Preferences lower priority but do not automatically mean “skip”.

Words before we use them

- Hard gate = explicit mandatory condition with no equivalent route stated.
- Preferred = desired but not stated as mandatory.
- Equivalent experience = employer wording allowing evidence other than a named degree.
- Apply/repair/skip = decision plus the next action, not an emotional judgment.

Useful visual / current reference

[VIDEO: Google India - organise job applications and prepare for interviews - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: linkedin_profile](#)

Use rule: The external source is visual/context support. Your saved artifact, query/code, explanation and evidence are authoritative for course assessment. If a video range is not independently bounded, the link stays but is labelled optional rather than invented.

Follow with me once

Sample A says “Bachelor’s degree in CS/IT or equivalent practical experience; SQL/Python required; Spark preferred.” A non-CS graduate with strong relevant projects can mark degree as “equivalent route explicitly allowed,” core SQL/Python as evidence-backed, Spark as proof available, and APPLY. Sample B says “B.Tech CSE/IT mandatory; campus 2026 batch only.” If you do not meet that hard gate, SKIP without treating it as a verdict on your career.

Try it yourself

Score the eight sample JDs in `practice/job_targeting/sample_job_descriptions.csv` using `score_jobs.py`, then apply the same rubric to five real roles you find yourself. Do not change a JD’s wording to make yourself eligible.

Check your work

- Decision names the exact hard gate or equivalent clause.
- Core skill match is supported by project evidence, not course completion alone.
- Logistics such as location/interview/shift are tracked separately.
- Every “repair” item becomes a small study or portfolio task.

Break case / common failure

A common failure is producing something that sounds polished but cannot survive one follow-up question. Another is hiding uncertainty instead of naming it. Save the weak first attempt, identify the unsupported claim or reasoning gap, and repair the smallest boundary.

Recovery ladder

1) Re-read the exact question/requirement. 2) State grain/contract/evidence before tools. 3) Remove claims you cannot prove. 4) Reproduce the smallest failing example. 5) Repair, rerun and reconcile. 6) Explain the corrected result in plain language without opening the answer file.

Fresh changed retry

Re-score one borderline job after removing all “preferred” requirements. Did your decision change for the right reason?

Revision in your own words

Degree-aware job targeting: apply/skip rules: truthful claim -> concrete evidence -> explicit validation -> known limitation -> changed retry. If the answer depends on “trust me,” it is not ready for interview use.

Evidence to keep

Keep the genuine first attempt plus the repaired version. Mastery means you can produce or explain the result without reading the learner book, and can answer at least one “why?”, one failure-mode question and one changed-input question.

DE19-05

SQL interview drills

Watch first

[1. Alex The Analyst - SQL Interview Questions and Answers for Beginners](#)

Optional visual - no verified bounded range

[2. Alex The Analyst - Solving Easy SQL Interview Questions](#)

02:33-06:57 first question; 06:58-11:43 second question

[3. learn by doing it - Data Engineer Interview Prep: SQL, Python & PySpark](#)

SQL 00:43-01:15:36; Python 01:15:37-02:05:19; other topics optional/unbounded

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 5 of 14 | Job-launch evidence, not note-copying | Normal mastery is untimed

Why this matters

SQL interviews test whether you can preserve grain and reason about results under pressure, not whether you memorized syntax. A query that returns a number can still be wrong because of fan-out, wrong filters or ties.

Picture it in your head

Before typing SQL, state one row of each table, the desired output grain and the control total. Then build the query in layers. The database becomes a reasoning tool rather than a syntax quiz.

Words before we use them

- Output grain = what one result row represents.
- Fan-out = unintended row multiplication after a join.
- Window function = calculation across related rows without collapsing all rows.
- Tie policy = explicit rule for equal ranking/timestamps.

Useful visual / current reference

[VIDEO: Alex The Analyst - SQL Interview Questions and Answers for Beginners - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: Alex The Analyst - Solving Easy SQL Interview Questions - Open video](#)

Watch: 02:33-06:57 first question; 06:58-11:43 second question

[VIDEO: learn by doing it - Data Engineer Interview Prep: SQL, Python & PySpark - Open video](#)

Watch: SQL 00:43-01:15:36; Python 01:15:37-02:05:19; other topics optional/unbounded

[OFFICIAL/CURRENT: postgres_select](#)

Use rule: The external source is visual/context support. Your saved artifact, query/code, explanation and evidence are authoritative for course assessment. If a video range is not independently bounded, the link stays but is labelled optional rather than invented.

Follow with me once

The supplied fixture asks for each customer's latest paid order. A strong answer first filters paid rows, ranks per customer by updated_at and a deterministic order_id tiebreaker, then chooses rank 1. It separately checks customer count and total paid amount so a plausible-looking answer is not accepted blindly.

Try it yourself

Complete the six prompts in practice/sql/SQL_INTERVIEW_DRILLS.md against interview_fixture.sqlite. Save your first query and result before opening protected solutions.

Check your work

- State grain before each query.
- Check row count/key uniqueness and one aggregate control.
- Explain why the join/ranking does not duplicate business entities.
- For ties, state the deterministic rule or deliberately surface ambiguity.

Break case / common failure

A common failure is producing something that sounds polished but cannot survive one follow-up question. Another is hiding uncertainty instead of naming it. Save the weak first attempt, identify the unsupported claim or reasoning gap, and repair the smallest boundary.

Recovery ladder

1) Re-read the exact question/requirement. 2) State grain/contract/evidence before tools. 3) Remove claims you cannot prove. 4) Reproduce the smallest failing example. 5) Repair, rerun and reconcile. 6) Explain the corrected result in plain language without opening the answer file.

Fresh changed retry

Solve one drill a second way (CTE vs subquery, GROUP BY vs window) and prove equivalent business results.

Revision in your own words

SQL interview drills: truthful claim -> concrete evidence -> explicit validation -> known limitation -> changed retry. If the answer depends on "trust me," it is not ready for interview use.

Evidence to keep

Keep the genuine first attempt plus the repaired version. Mastery means you can produce or explain the result without reading the learner book, and can answer at least one “why?”, one failure-mode question and one changed-input question.

DE19-06

Python interview drills

Watch first

[1. learn by doing it - Data Engineer Interview Prep: SQL, Python & PySpark](#)

SQL 00:43-01:15:36; Python 01:15:37-02:05:19; other topics optional/unbounded

[2. freeCodeCamp - Data Structure Patterns for LeetCode Interviews](#)

00:00-03:10 arrays; 03:11-04:55 strings; 10:02-15:53 hashmap; 15:54-18:51 hashmap practice

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 6 of 14 | Job-launch evidence, not note-copying | Normal mastery is untimed

Why this matters

Python interview questions are usually small on purpose. The interviewer wants to see decomposition, data-structure choice, edge cases, complexity and communication - not a 300-line framework.

Picture it in your head

Use a four-step loop: restate the input/output, show a tiny example, choose the simplest data structure, then code and test edge cases. Narrate decisions without talking over every keystroke.

Words before we use them

- Hash map = key/value structure with average $O(1)$ lookup.
- Set = unique membership structure.
- Edge case = valid input that stresses assumptions such as empty, duplicate or missing data.
- Complexity = how work/memory grows with input size.

Useful visual / current reference

[VIDEO: learn by doing it - Data Engineer Interview Prep: SQL, Python & PySpark - Open video](#)

Watch: SQL 00:43-01:15:36; Python 01:15:37-02:05:19; other topics optional/unbounded

[VIDEO: freeCodeCamp - Data Structure Patterns for LeetCode Interviews - Open video](#)

Watch: 00:00-03:10 arrays; 03:11-04:55 strings; 10:02-15:53 hashmap; 15:54-18:51 hashmap practice

[OFFICIAL/CURRENT: python_tutorial](#)

Use rule: The external source is visual/context support. Your saved artifact, query/code, explanation and evidence are authoritative for course assessment. If a video range is not independently bounded, the link stays but is labelled optional rather than invented.

Follow with me once

For “first duplicate order ID,” a set is enough: scan left-to-right; if ID already seen, return it; otherwise add it. Time $O(n)$, space $O(n)$. A nested-loop answer can work but is $O(n^2)$. The important part is explaining why first-seen order matters.

Try it yourself

Implement the five functions in `practice/python/python_interview_drills.py`. Run `test_python_interview_drills.py`. Keep the first failing output, repair, then explain one complexity trade-off.

Check your work

- Functions handle empty input, duplicates and missing values specified in the prompt.
- Tests pass because behavior is correct, not because tests were weakened.
- You can explain the chosen data structure in one sentence.
- Code is readable without clever one-liners.

Break case / common failure

A common failure is producing something that sounds polished but cannot survive one follow-up question. Another is hiding uncertainty instead of naming it. Save the weak first attempt, identify the unsupported claim or reasoning gap, and repair the smallest boundary.

Recovery ladder

1) Re-read the exact question/requirement. 2) State grain/contract/evidence before tools. 3) Remove claims you cannot prove. 4) Reproduce the smallest failing example. 5) Repair, rerun and reconcile. 6) Explain the corrected result in plain language without opening the answer file.

Fresh changed retry

Rewrite one solved function with a deliberately worse data structure, compare complexity, then restore the better version.

Revision in your own words

Python interview drills: truthful claim -> concrete evidence -> explicit validation -> known limitation -> changed retry. If the answer depends on “trust me,” it is not ready for interview use.

Evidence to keep

Keep the genuine first attempt plus the repaired version. Mastery means you can produce or explain the result without reading the learner book, and can answer at least one “why?”, one failure-mode question and one changed-input question.

Make the latest-version answer independent of input order

Suppose A has version 1 with value 3, version 1 with value 4, and version 2 with value 8. This interview task asks you to reject a conflict at the maximum version. The answer is therefore A/version 2/value 8, whichever order the rows arrive in. A one-pass function that rejects the first version-1 disagreement would fail this contract. First find the maximum version per ID, then compare only rows at that maximum. A different ingestion contract may reject conflicts at any version; state which question you are solving.

DE19-07

Data-engineering concepts and troubleshooting drills

Watch first

[1. learn by doing it - Data Engineer Interview Prep: SQL, Python & PySpark](#)

SQL 00:43-01:15:36; Python 01:15:37-02:05:19; other topics optional/unbounded

[2. Afaque Ahmad - How I Mastered Data Engineering System Design Interviews](#)

04:19-13:21 six-step framework; 13:22-27:32 requirements; 38:38-47:20 batch vs streaming; 01:29:02-01:37:36 quality/observability

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 7 of 14 | Job-launch evidence, not note-copying | Normal mastery is untimed

Why this matters

Troubleshooting questions reveal whether you understand data engineering as a system. A junior candidate is not expected to know every product, but should be able to protect data, narrow the failure boundary, and choose evidence before guessing.

Picture it in your head

For any incident ask: what should have happened, what durable state is safe, where did actual behavior first diverge, and what evidence can falsify the leading hypothesis?

Words before we use them

- Failure domain = component/boundary that can fail independently.
- Invariant = property that must stay true across success/retry/failure.
- Checkpoint = durable progress marker whose ownership must be clear.
- Reconciliation = independent comparison proving business completeness/correctness.

Useful visual / current reference

[VIDEO: learn by doing it - Data Engineer Interview Prep: SQL, Python & PySpark - Open video](#)

Watch: SQL 00:43-01:15:36; Python 01:15:37-02:05:19; other topics optional/unbounded

[VIDEO: Afaque Ahmad - How I Mastered Data Engineering System Design Interviews - Open video](#)

Watch: 04:19-13:21 six-step framework; 13:22-27:32 requirements; 38:38-47:20 batch vs streaming; 01:29:02-01:37:36 quality/observability

Use rule: The external source is visual/context support. Your saved artifact, query/code, explanation and evidence are authoritative for course assessment. If a video range is not

independently bounded, the link stays but is labelled optional rather than invented.

Follow with me once

Scenario: source API returned 5,000 rows, sink has 4,970, checkpoint advanced. Do not begin by “rerunning everything.” Freeze progress, compare source IDs vs sink IDs, inspect reject/quarantine and commit sequence, identify whether 30 records failed before or after durable write, repair checkpoint ownership, then replay only the unsafe range.

Try it yourself

Answer the six incident cards in `practice/troubleshooting/INCIDENT_CARDS.md` using the evidence-first template. Each answer must include safe state, first three checks, likely boundary, recovery and prevention.

Check your work

- No destructive recovery before source/sink/state evidence is saved.
- Checkpoint/offset advancement is tied to durable validated work.
- Bad data and infrastructure failure are separated.
- Recovery ends with reconciliation, not just “job succeeded”.

Break case / common failure

A common failure is producing something that sounds polished but cannot survive one follow-up question. Another is hiding uncertainty instead of naming it. Save the weak first attempt, identify the unsupported claim or reasoning gap, and repair the smallest boundary.

Recovery ladder

1) Re-read the exact question/requirement. 2) State grain/contract/evidence before tools. 3) Remove claims you cannot prove. 4) Reproduce the smallest failing example. 5) Repair, rerun and reconcile. 6) Explain the corrected result in plain

language without opening the answer file.

Fresh changed retry

Take one incident and assume your first hypothesis is wrong. Name the evidence that would prove it wrong and the next hypothesis.

Revision in your own words

Data-engineering concepts and troubleshooting drills: truthful claim -> concrete evidence -> explicit validation -> known limitation -> changed retry. If the answer depends on “trust me,” it is not ready for interview use.

Evidence to keep

Keep the genuine first attempt plus the repaired version. Mastery means you can produce or explain the result without reading the learner book, and can answer at least one “why?”, one failure-mode question and one changed-input question.

DE19-08

Project walkthrough interview

Watch first

[1. Afaque Ahmad - How I Mastered Data Engineering System Design Interviews](#)

04:19-13:21 six-step framework; 13:22-27:32 requirements; 38:38-47:20 batch vs streaming; 01:29:02-01:37:36 quality/observability

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 8 of 14 | Job-launch evidence, not note-copying | Normal mastery is untimed

Why this matters

“Walk me through a project” is a reasoning interview, not a technology recital. The interviewer is testing whether you understand the problem, your decisions, failure boundaries, validation and personal contribution.

Picture it in your head

Use a seven-part story: business need → source/grain → architecture → one difficult decision → failure/recovery → validation → limitation/next step. Keep a 90-second version and a deeper 5-minute version.

Words before we use them

- Trade-off = benefit accepted with a cost/limitation.
- Personal contribution = what you actually designed/built/tested.
- Evidence = artifact that supports the claim.
- Limitation = known boundary honestly acknowledged.

Useful visual / current reference

[VIDEO: Afaque Ahmad - How I Mastered Data Engineering System Design Interviews - Open video](#)

Watch: 04:19-13:21 six-step framework; 13:22-27:32 requirements; 38:38-47:20 batch vs streaming; 01:29:02-01:37:36 quality/observability

[OFFICIAL/CURRENT: github_resume](#)

Use rule: The external source is visual/context support. Your saved artifact, query/code, explanation and evidence are authoritative for course assessment. If a video range is not independently bounded, the link stays but is labelled optional rather than invented.

Follow with me once

ParcelFlow walkthrough: “The goal was a replay-safe daily orders pipeline. One source row represented an order version. I preserved immutable raw bytes before validation, quarantined invalid foreign keys/quantities, committed incremental progress only after durable trusted output, reconciled source/accepted/quarantine counts, and injected a write failure to prove replay. I used local fixtures, not a production company workload; scale/cloud deployment remain design extensions.”

Try it yourself

Fill practice/project_defense/PROJECT_WALKTHROUGH_TEMPLATE.md for two M18 projects. Record yourself twice: 90 seconds and 5 minutes. Do not read the README during the second recording.

Check your work

- Problem and grain appear before tool list.
- At least one decision includes an alternative and trade-off.
- Failure/recovery is specific and evidence-backed.
- You distinguish local project evidence from unexecuted production/cloud claims.

Break case / common failure

A common failure is producing something that sounds polished but cannot survive one follow-up question. Another is hiding uncertainty instead of naming it. Save the weak first attempt, identify the unsupported claim or reasoning gap, and repair the smallest boundary.

Recovery ladder

1) Re-read the exact question/requirement. 2) State grain/contract/evidence before tools. 3) Remove claims you cannot prove. 4) Reproduce the smallest failing example. 5) Repair, rerun and reconcile. 6) Explain the corrected result in plain

language without opening the answer file.

Fresh changed retry

Let a reviewer interrupt after any sentence with “why?” or “how do you know?”. Repair answers that collapse into “because the tutorial said so”.

Revision in your own words

Project walkthrough interview: truthful claim -> concrete evidence -> explicit validation -> known limitation -> changed retry. If the answer depends on “trust me,” it is not ready for interview use.

Evidence to keep

Keep the genuine first attempt plus the repaired version. Mastery means you can produce or explain the result without reading the learner book, and can answer at least one “why?”, one failure-mode question and one changed-input question.

DE19-09

Architecture/system-design defense for junior roles

Watch first

[1. Afaque Ahmad - How I Mastered Data Engineering System Design Interviews](#)

04:19-13:21 six-step framework; 13:22-27:32 requirements; 38:38-47:20 batch vs streaming; 01:29:02-01:37:36 quality/observability

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 9 of 14 | Job-launch evidence, not note-copying | Normal mastery is untimed

Why this matters

Junior system-design interviews usually reward structure and trade-off reasoning more than obscure distributed-systems trivia. You should clarify requirements, estimate scale enough to choose a shape, draw the major data path, then discuss correctness/failure/security/cost.

Picture it in your head

Draw left-to-right: producers/sources → ingestion → durable raw → processing → trusted storage/serving → consumers. Draw state/checkpoints separately. Then attack the design with volume, lateness, schema change and component failure.

Words before we use them

- Functional requirement = what the system must do.
- Non-functional requirement = latency, volume, reliability, cost/security constraints.
- Backpressure = downstream cannot keep up with incoming work.
- ADR = short record of an architectural decision and its consequences.

Useful visual / current reference

[VIDEO: Afaque Ahmad - How I Mastered Data Engineering System Design Interviews - Open video](#)

Watch: 04:19-13:21 six-step framework; 13:22-27:32 requirements; 38:38-47:20 batch vs streaming; 01:29:02-01:37:36 quality/observability

Use rule: The external source is visual/context support. Your saved artifact, query/code, explanation and evidence are authoritative for course assessment. If a video range is not independently bounded, the link stays but is labelled optional rather than invented.

Follow with me once

Prompt: design order analytics updated within 15 minutes. Clarify order volume and acceptable lateness. Choose micro-batch unless per-second decisions are required. Preserve raw object storage, validate into Silver/Delta, create serving marts, orchestrate/checkpoint, reconcile order counts/revenue, monitor freshness, and explain how backfill avoids double counting.

Try it yourself

Solve two cases in practice/architecture/ARCHITECTURE_CASES.md. Use the scorecard: requirements, grain, scale, flow, state, quality, failure, security, cost and trade-offs.

Check your work

- Architecture choices follow requirements rather than tool preference.
- Durable raw/replay path is explicit.
- You state at least one failure domain and recovery.
- Cost/security/governance are included before the interviewer asks.

Break case / common failure

A common failure is producing something that sounds polished but cannot survive one follow-up question. Another is hiding uncertainty instead of naming it. Save the weak first attempt, identify the unsupported claim or reasoning gap, and repair the smallest boundary.

Recovery ladder

1) Re-read the exact question/requirement. 2) State grain/contract/evidence before tools. 3) Remove claims you cannot prove. 4) Reproduce the smallest failing example. 5) Repair, rerun and reconcile. 6) Explain the corrected result in plain language without opening the answer file.

Fresh changed retry

Change latency from daily to 30 seconds. Redesign only the parts that truly need to change and defend why the rest stays.

Revision in your own words

Architecture/system-design defense for junior roles: truthful claim -> concrete evidence -> explicit validation -> known limitation -> changed retry. If the answer depends on “trust me,” it is not ready for interview use.

Evidence to keep

Keep the genuine first attempt plus the repaired version. Mastery means you can produce or explain the result without reading the learner book, and can answer at least one “why?”, one failure-mode question and one changed-input question.

DE19-10

Behavioral answers and career-transition story

Watch first

[1. Indeed - Behavioral Interview Questions and Answers: Use the STAR Technique](#)

00:23-01:24 behavioral questions; 01:25-03:09 STAR structure; 05:04-06:09 worked difficult-problem answer; 06:10-08:24 best practices

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 10 of 14 | Job-launch evidence, not note-copying | Normal mastery is untimed

Why this matters

Behavioral questions are evidence questions. A strong answer is not a personality speech; it is a specific story showing how you approached a problem, what you did, and what changed. Career-transition candidates can use education, projects, teamwork and real responsibilities without inventing employment.

Picture it in your head

Use STAR as a compression tool: Situation gives only necessary context; Task says your responsibility; Action gets most airtime; Result names evidence/learning. Add a final reflection when the result was imperfect.

Words before we use them

- STAR = Situation, Task, Action, Result.
- Ownership = actions you personally took, not what “we” vaguely did.
- Reflection = what you learned or would change.
- Transition story = concise explanation linking previous background, deliberate skill-building and target role.

Useful visual / current reference

[VIDEO: Indeed - Behavioral Interview Questions and Answers: Use the STAR Technique - Open video](#)

Watch: 00:23-01:24 behavioral questions; 01:25-03:09 STAR structure; 05:04-06:09 worked difficult-problem answer; 06:10-08:24 best practices

[OFFICIAL/CURRENT: ut_behavior](#)

Use rule: The external source is visual/context support. Your saved artifact, query/code, explanation and evidence are authoritative for course assessment. If a video range is not independently bounded, the link stays but is labelled optional rather than invented.

Follow with me once

Question: “Tell me about a failure.” Use a real project event: a consumer originally committed progress after a sink failure. Situation: reliability exercise. Task: guarantee recoverability. Action: reproduced the failure with a controlled fake client, separated poison data from infrastructure exceptions, moved commit after durable write, added replay test. Result: failed valid writes stayed uncommitted and replay preserved uniqueness. Reflection: exception taxonomy is part of data correctness.

Try it yourself

Build six truthful stories in practice/behavioral/STAR_STORY_BANK.md: failure, ambiguity, learning, disagreement/feedback, ownership, and prioritization. Draft a 60-second transition story that never apologizes for your degree and never invents experience.

Check your work

- Stories are specific enough that dates/project context make sense.
- Action uses “I” for your contribution.
- Result is factual even if modest.
- Transition story explains what you did to become job-ready and points to proof.

Break case / common failure

A common failure is producing something that sounds polished but cannot survive one follow-up question. Another is hiding uncertainty instead of naming it. Save the weak first attempt, identify the unsupported claim or reasoning gap, and repair the smallest boundary.

Recovery ladder

1) Re-read the exact question/requirement. 2) State grain/contract/evidence before tools. 3) Remove claims you cannot prove. 4) Reproduce the smallest failing example. 5) Repair, rerun and reconcile. 6) Explain the corrected result in plain language without opening the answer file.

Fresh changed retry

Cut one 3-minute story to 90 seconds without deleting the decision or evidence.

Revision in your own words

Behavioral answers and career-transition story: truthful claim -> concrete evidence -> explicit validation -> known limitation -> changed retry. If the answer depends on “trust me,” it is not ready for interview use.

Evidence to keep

Keep the genuine first attempt plus the repaired version. Mastery means you can produce or explain the result without reading the learner book, and can answer at least one “why?”, one failure-mode question and one changed-input question.

DE19-11

Technical English: explain code, failures and decisions clearly

Watch first

[1. Indeed - Behavioral Interview Questions and Answers: Use the STAR Technique](#)

00:23-01:24 behavioral questions; 01:25-03:09 STAR structure; 05:04-06:09 worked difficult-problem answer; 06:10-08:24 best practices

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 11 of 14 | Job-launch evidence, not note-copying | Normal mastery is untimed

Why this matters

Technical English is not about sounding native. It is about making technical reasoning easy to follow: clear subject, concrete nouns, short causal chains, explicit uncertainty, and a clean handoff between problem, action and result.

Picture it in your head

Use “Claim → because → evidence → next step.” Example: “I think the loss happened after validation because source IDs are complete but 30 sink IDs are missing. I would inspect the write/commit boundary next.”

Words before we use them

- Signpost = phrase such as “first”, “the key assumption”, “the failure boundary”.
- Hedge = honest uncertainty such as “my leading hypothesis is...”.
- Causal chain = A caused B, evidenced by C.
- Teach-back = explain a concept without copying its definition.

Useful visual / current reference

[VIDEO: Indeed - Behavioral Interview Questions and Answers: Use the STAR Technique - Open video](#)

Watch: 00:23-01:24 behavioral questions; 01:25-03:09 STAR structure; 05:04-06:09 worked difficult-problem answer; 06:10-08:24 best practices

[OFFICIAL/CURRENT: mit_interview](#)

Use rule: The external source is visual/context support. Your saved artifact, query/code, explanation and evidence are authoritative for course assessment. If a video range is not independently bounded, the link stays but is labelled optional rather than invented.

Follow with me once

Instead of “Airflow failed so I fixed DAG and then it worked,” say: “The DAG imported, but the task sensor never found the dated input because the second terminal lost AIRFLOW_HOME and the fixture path differed from the DAG path. I restored a reusable activation script, moved fixtures to learner-owned paths, reran a supplied historical partition, and verified output plus logs.”

Try it yourself

Use `practice/english/TECHNICAL_ENGLISH_DRILLS.md`. Record five 60-90 second answers: explain a join fan-out, checkpoint bug, schema drift, architecture trade-off and project failure. Transcribe one recording and remove vague pronouns/filler.

Check your work

- Sentence names the object (“checkpoint”, “sink write”, “order grain”) rather than “it/thing”.
- Uncertainty is explicit; you do not present hypotheses as facts.
- Answer has beginning/problem, reasoning/action, result/next step.
- Accent is irrelevant; clarity and technical truth are the pass criteria.

Break case / common failure

A common failure is producing something that sounds polished but cannot survive one follow-up question. Another is hiding uncertainty instead of naming it. Save the weak first attempt, identify the unsupported claim or reasoning gap, and repair the smallest boundary.

Recovery ladder

1) Re-read the exact question/requirement. 2) State grain/contract/evidence before tools. 3) Remove claims you cannot prove. 4) Reproduce the smallest failing example. 5) Repair, rerun and reconcile. 6) Explain the corrected result in plain language without opening the answer file.

Fresh changed retry

Explain the same incident to (a) an engineer and (b) a nontechnical manager. Keep the facts constant; change only depth and vocabulary.

Revision in your own words

Technical English: explain code, failures and decisions clearly: truthful claim -> concrete evidence -> explicit validation -> known limitation -> changed retry. If the answer depends on “trust me,” it is not ready for interview use.

Evidence to keep

Keep the genuine first attempt plus the repaired version. Mastery means you can produce or explain the result without reading the learner book, and can answer at least one “why?”, one failure-mode question and one changed-input question.

DE19-12

Application tracker, feedback loop and weekly repair cycle

Watch first

[1. Google India - organise job applications and prepare for interviews](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 12 of 14 | Job-launch evidence, not note-copying | Normal mastery is untimed

Why this matters

Job search improves when it becomes an observable process rather than a daily mood. Track applications, response stages and recurring failure signals; then repair one bottleneck at a time.

Picture it in your head

Treat the job hunt like a funnel: targeted roles → applications → screens → technical rounds → final rounds → offers. Low conversion at each stage suggests a different repair: targeting/resume, communication, technical proof or interview execution.

Words before we use them

- Conversion rate = proportion moving from one stage to the next.
- Feedback signal = evidence from outcomes or interviewer comments.
- Repair cycle = one weekly hypothesis and measurable adjustment.
- Vanity metric = large application count without quality/fit evidence.

Useful visual / current reference

[VIDEO: Google India - organise job applications and prepare for interviews - Open video](#)

Watch: Optional visual - no verified bounded range

Use rule: The external source is visual/context support. Your saved artifact, query/code, explanation and evidence are authoritative for course assessment. If a video range is not independently bounded, the link stays but is labelled optional rather than invented.

Follow with me once

If 40 carefully targeted applications produce zero screens, do not immediately study more Spark. Audit hard-gate targeting, resume claims, project links and keyword fit. If screens are frequent but technical rounds fail, shift the weekly repair to SQL/Python/troubleshooting evidence.

Try it yourself

Use `practice/job_search/APPLICATION_TRACKER.csv` and `weekly_review.py`. Record every application truthfully. At week end, choose exactly one bottleneck, one repair experiment and one metric for the next week.

Check your work

- Tracker has role URL/company/date/stage/hard-gate decision and next action.
- No fake recruiter/interview status.
- Weekly repair is linked to a funnel signal, not random course hopping.
- Rejected roles remain useful data but do not become personal judgments.

Break case / common failure

A common failure is producing something that sounds polished but cannot survive one follow-up question. Another is hiding uncertainty instead of naming it. Save the weak first attempt, identify the unsupported claim or reasoning gap, and repair the smallest boundary.

Recovery ladder

1) Re-read the exact question/requirement. 2) State grain/contract/evidence before tools. 3) Remove claims you cannot prove. 4) Reproduce the smallest failing example. 5) Repair, rerun and reconcile. 6) Explain the corrected result in plain language without opening the answer file.

Fresh changed retry

Run the weekly review on the supplied sample tracker, then create a second scenario where screens are high but coding-round conversion is low. Choose a different repair.

Revision in your own words

Application tracker, feedback loop and weekly repair cycle: truthful claim -> concrete evidence -> explicit validation -> known limitation -> changed retry. If the answer depends on “trust me,” it is not ready for interview use.

Evidence to keep

Keep the genuine first attempt plus the repaired version. Mastery means you can produce or explain the result without reading the learner book, and can answer at least one “why?”, one failure-mode question and one changed-input question.

DE19-13

Timed coding test: arrays, strings and hash-map problems

Watch first

[1. freeCodeCamp - Data Structure Patterns for LeetCode Interviews](#)

00:00-03:10 arrays; 03:11-04:55 strings; 10:02-15:53 hashmap; 15:54-18:51 hashmap practice

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 13 of 14 | Job-launch evidence, not note-copying | Normal mastery is untimed

Why this matters

Some junior interviews include short coding exercises. The course rule still applies: master the pattern untimed first, then optionally add interview timing. Speed before correctness trains panic and shallow memorization.

Picture it in your head

Use a repeatable interview loop: clarify → examples → brute force → better data structure → code → tests → complexity. Arrays, strings, sets and hash maps cover many entry-level patterns.

Words before we use them

- Two-pass = collect information then use it.
- Frequency map = hashmap from value to count.
- Two pointers = two indices moving with an invariant.
- Optional timing = second-stage simulation after untimed mastery; it is not the learning pass criterion.

Useful visual / current reference

[VIDEO: freeCodeCamp - Data Structure Patterns for LeetCode Interviews - Open video](#)

Watch: 00:00-03:10 arrays; 03:11-04:55 strings; 10:02-15:53 hashmap; 15:54-18:51 hashmap practice

[OFFICIAL/CURRENT: python_tutorial](#)

Use rule: The external source is visual/context support. Your saved artifact, query/code, explanation and evidence are authoritative for course assessment. If a video range is not independently bounded, the link stays but is labelled optional rather than invented.

Follow with me once

Prompt: return the first non-repeating character. First build counts in $O(n)$; second scan in original order and return the first count 1. State empty-string behavior. Do not optimize into unreadable cleverness.

Try it yourself

Solve the six tasks in `practice/coding_test/CODING_TEST_TASKS.md` untimed. Run tests. After mastery, optionally repeat three different variants in a self-chosen interview window and record where reasoning - not typing - slowed you.

Check your work

- All supplied cases pass without modifying expected results.
- You can explain $O(n)/O(n^2)$ choices.
- Empty/duplicate/no-solution cases are explicit.
- Timed attempt is optional and comes after untimed correct proof.

Break case / common failure

A common failure is producing something that sounds polished but cannot survive one follow-up question. Another is hiding uncertainty instead of naming it. Save the weak first attempt, identify the unsupported claim or reasoning gap, and repair the smallest boundary.

Recovery ladder

1) Re-read the exact question/requirement. 2) State grain/contract/evidence before tools. 3) Remove claims you cannot prove. 4) Reproduce the smallest failing example. 5) Repair, rerun and reconcile. 6) Explain the corrected result in plain language without opening the answer file.

Fresh changed retry

Change a list problem into a streaming iterator where you cannot sort or rewind. Redesign the state.

Revision in your own words

Timed coding test: arrays, strings and hash-map problems: truthful claim -> concrete evidence -> explicit validation -> known limitation -> changed retry. If the answer depends on “trust me,” it is not ready for interview use.

Evidence to keep

Keep the genuine first attempt plus the repaired version. Mastery means you can produce or explain the result without reading the learner book, and can answer at least one “why?”, one failure-mode question and one changed-input question.

DE19-14

Combined SQL + Python interview simulation

Watch first

[1. Alex The Analyst - Solving Easy SQL Interview Questions](#)

02:33-06:57 first question; 06:58-11:43 second question

[2. freeCodeCamp - Data Structure Patterns for LeetCode Interviews](#)

00:00-03:10 arrays; 03:11-04:55 strings; 10:02-15:53 hashmap; 15:54-18:51 hashmap practice

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 14 of 14 | Job-launch evidence, not note-copying | Normal mastery is untimed

Why this matters

Real assessments often mix SQL extraction with Python post-processing or validation. The final module lesson tests context switching while preserving the same discipline: define grain, save expected controls, write the smallest correct solution, and reconcile.

Picture it in your head

SQL should return a well-defined intermediate dataset; Python should consume that contract, not silently reinterpret it. If the SQL grain is wrong, no Python cleanup can restore missing business meaning.

Words before we use them

- Interface contract = agreed columns/types/grain between SQL and Python.
- Intermediate result = explicit dataset handed between stages.
- Reconciliation = independent controls across both stages.
- Cold attempt = saved first attempt before review.

Useful visual / current reference

[VIDEO: Alex The Analyst - Solving Easy SQL Interview Questions - Open video](#)

Watch: 02:33-06:57 first question; 06:58-11:43 second question

[VIDEO: freeCodeCamp - Data Structure Patterns for LeetCode Interviews - Open video](#)

Watch: 00:00-03:10 arrays; 03:11-04:55 strings; 10:02-15:53 hashmap; 15:54-18:51 hashmap practice

[OFFICIAL/CURRENT: postgres_select](#)

[OFFICIAL/CURRENT: python_tutorial](#)

Use rule: The external source is visual/context support. Your saved artifact, query/code, explanation and evidence are authoritative for course assessment. If a video range is not independently bounded, the link stays but is labelled optional rather than invented.

Follow with me once

The supplied simulation asks SQL for paid-order customer totals and Python for risk flags. First state one SQL row per customer. Reconcile total paid amount before exporting. Python reads the exported rows, flags high value and duplicate-account conditions, and verifies output row count equals customer count.

Try it yourself

Complete practice/combined_simulation/README_TASK.md with learner SQL and Python files. Save commands/results. Only after a genuine attempt open the protected solution/reference. Then take G07 Gate A cold.

Check your work

- SQL intermediate grain is explicit and unique.
- Aggregate totals reconcile to source controls.
- Python never changes business totals while adding derived flags.
- First attempt, repair and changed retry are saved separately.

Break case / common failure

A common failure is producing something that sounds polished but cannot survive one follow-up question. Another is hiding uncertainty instead of naming it. Save the weak first attempt, identify the unsupported claim or reasoning gap, and repair the smallest boundary.

Recovery ladder

1) Re-read the exact question/requirement. 2) State grain/contract/evidence before tools. 3) Remove claims you cannot prove. 4) Reproduce the smallest failing example. 5) Repair, rerun and reconcile. 6) Explain the corrected result in plain language without opening the answer file.

Fresh changed retry

Add one new paid order tied at the existing maximum timestamp and prove your SQL/Python contract does not skip or double count it.

Revision in your own words

Combined SQL + Python interview simulation: truthful claim -> concrete evidence -> explicit validation -> known limitation -> changed retry. If the answer depends on “trust me,” it is not ready for interview use.

Evidence to keep

Keep the genuine first attempt plus the repaired version. Mastery means you can produce or explain the result without reading the learner book, and can answer at least one “why?”, one failure-mode question and one changed-input question.